

Concepts for multi-cluster deployments (v2)

26.7.0

Understand multi-cluster deployment with synchronous replication without an external Infinispan cluster.



This guide is describing a feature which is currently in preview. Please provide your feedback by [joining this discussion](#) while we’re continuing to work on this.

This topic describes a highly available multi-cluster setup and the behavior to expect. It outlines the requirements of the high availability architecture and describes the benefits and tradeoffs. See [Multi-cluster deployments \(v2\)](#) for the introduction.

When to use this setup

Use this setup to provide Keycloak deployments that are able to tolerate cluster failures, reducing the likelihood of downtime.

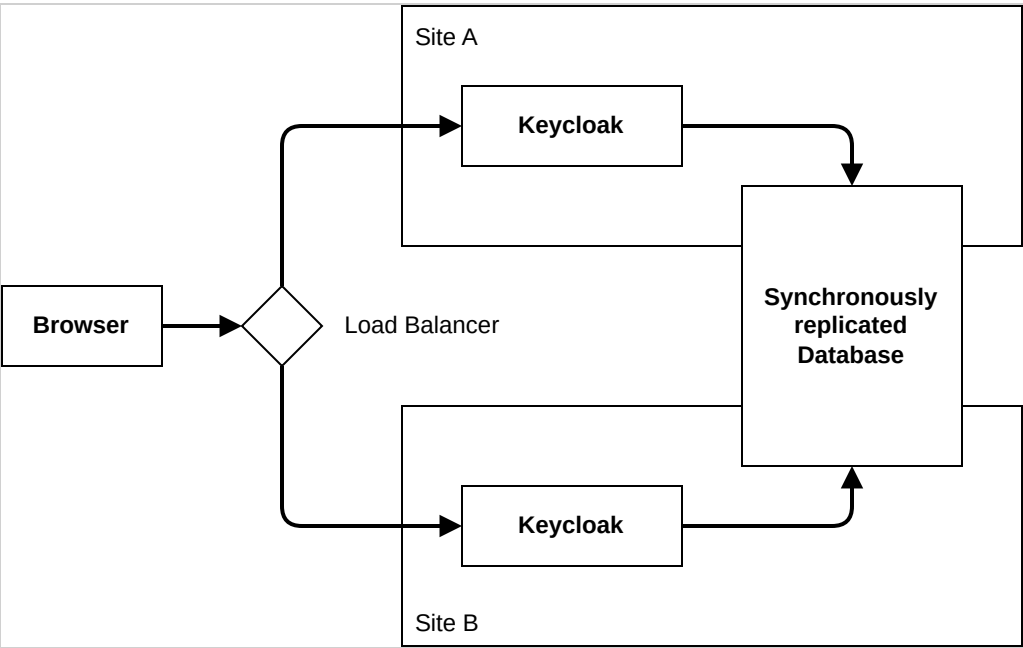
Unlike the standard [Concepts for multi-cluster deployments](#), this setup does not require an external Infinispan cluster. This simplifies the deployment architecture and removes the need for an external monitoring solution that reconfigures the external Infinispan on failover. It also eliminates operational procedures on failback.

Deployment, data storage and caching

Two or more independent Keycloak deployments running in different sites.

Users, realms, clients, sessions, and other entities are stored in a database that is replicated synchronously across all sites.

Realm data (including clients and groups) and authorization data are cached in the Keycloak embedded Infinispan caches as local caches.



Background jobs

Background jobs will run only in one of the active clusters and will be automatically redistributed to another cluster on a cluster failure.

Updatesto realm data and cache invalidation

On this page

[When to use this setup](#)

[Deployment, data storage and caching](#)

[Causes of data and service loss](#)

[Failures which this setup can survive](#)

[Known limitations](#)

[Questions and answers](#)

[Next steps](#)

[✎ Edit this guide](#)



This paragraph describes the internal implementation of how cache invalidation works across clusters with the `stateless` feature to allow administrators to understand the emerging observed behavior. This internal implementation might change between minor or patch releases. If it changes, this section will be updated.

When the realm data, including clients and groups, is changed in one Keycloak instance, that data is updated in the database. The instance sends a cache invalidation message to the other sites via the database, leveraging the outbox pattern with a queuing table. The other sites retrieve the message via polling. The polling interval is configurable and set to 100ms by default. The sending instance waits for five times the polling interval for the messages to be consumed by the receiver before returning to the caller. This allows callers in regular operations to ensure that all nodes have received the invalidation message and will use the new data from the database upon the next call.

Updates to realm data in these multi-cluster v2 setups are slower than in single-cluster setups, as the invalidation mechanism adds a delay of up to 100 ms to calls that modify realm-related data.

When planning for large batch updates of realm data, consider shutting down all clusters except for one to avoid the delay, and start up the other clusters once the update is complete.

Invalidation messages are queued for up to 60 seconds, which is in line with the cluster membership mechanism, which updates its information in the database every 30–45 seconds.

To handle edge cases and to avoid permanently out-of-sync caches, any cached realm information expires by default after one hour. The data is then fetched again from the database on the next call that requires it.

Causes of data and service loss

While this setup aims for high availability, the following situations can still lead to service or data loss:

- Keycloak site failure may result in requests failing in the period between the failure and the load balancer detecting it, as requests may still be routed to the failed site.
- Once failures occur in the communication between the sites, manual steps may be necessary depending on the database vendor, for example to promote an instance to be the new writer.
- Degraded setups can lead to service or data loss if additional components fail. Monitoring is necessary to detect degraded setups.

Failures which this setup can survive

Failure	Recovery	RPO ¹	RT ²
Database node	If the writer instance fails, the database can promote a reader instance in the same or other site to be the new writer.	No data loss	Seconds to minutes (depending on the database)
Keycloak node	Multiple Keycloak instances run in each site. If one instance fails, some incoming requests might receive an error message or be delayed for some seconds until the load balancer takes the node out of rotation.	No data loss	Seconds to minutes (depending on the load balancer)

Failure	Recovery	RPO ¹	RT ²
Database connectivity loss	If the connectivity between sites is lost, the synchronous replication will fail. Some requests might receive an error message or be delayed for a few seconds. Manual operations might be necessary depending on the database. Keycloak instances that have access to a writable database instance will continue to serve traffic, while other nodes will mark themselves as not ready to the load balancer.	No data loss	Seconds to minutes (depending on the database and load balancer)
Site failure	If none of the Keycloak nodes are available, the load balancer will detect the outage and redirect the traffic to the other site. Some requests might receive an error message until the load balancer detects the failure.	No data loss	Seconds to minutes (depending on the load balancer)

Table footnotes:

¹ Tested Recovery Point Objective, assuming all parts of the setup were healthy at the time this occurred.

² Estimated Recovery Time.

The statement “No data loss” depends on the setup not being degraded from previous failures, which includes completing any pending manual operations to resynchronize the state between the sites.

To restore the service, restart any failed Keycloak node and restore its database connectivity. No other recovery operations are necessary for Keycloak. A standard load balancer configuration should detect the active Keycloak nodes automatically and route traffic to them. Recovery operations on the database side might depend on the database vendor.

Known limitations

Site Failure

A successful failover requires a setup not degraded from previous failures. Use monitoring to ensure degradations are detected and handled in a timely manner.

Questions and answers

Why synchronous database replication?

A synchronously replicated database ensures that data written in one site is always available in the other site after site failures and no data is lost. It also ensures that the next request will not return stale data, independent of which site serves it.

Why is a low-latency network between sites needed?

Synchronous replication defers the response to the caller until the data is received at the other site. A low latency is necessary as each request can potentially involve multiple interactions between the sites when data is updated, which would amplify the latency.

Is a synchronous cluster less stable than an asynchronous cluster?

An asynchronous setup would handle network failures between the sites gracefully, while the synchronous setup would delay requests and throw errors to the caller where the asynchronous setup would have deferred the writes to the database on the other site. However, as the sites would never be fully up-to-date, this setup could lead to data loss during failures. This would include:

- Lost changes leading to users being able to log in with an old password because database changes are not replicated to the other site at the point of failure when using an asynchronous database.

Therefore, tradeoffs exist between high availability and consistency. The focus of this topic is to prioritize consistency over availability with Keycloak.

Next steps

Continue reading the blueprint at [Deploying Keycloak for HA with the Operator \(v2\)](#).



© Keycloak Authors 2026. © 2026 The Linux Foundation. All rights reserved. The Linux Foundation has registered trademarks and uses trademarks. For a list of trademarks of The Linux Foundation, please see our [Trademark Usage page](#).